Voice Clinic Agent: Documentation and Knowledge Transfer

Version 2.0 · Prepared 17 October 2026 · Audience: developers, maintainers, reviewers and new team members

1. Purpose and scope

The Voice Clinic Agent lets a patient phone a clinic and talk to an AI receptionist named Asha. Asha can book, check, reschedule and cancel doctor appointments, register new patients, arrange follow-up consultations, and read out a patient's recent visit history. All data lives in MongoDB. Every component is open source and runs locally, so there are no per-call API fees.

This document explains what the system does, how it is built, how to run and operate it, where each responsibility lives in the code, what its known weaknesses are, and how to extend it. It is written so that a new engineer can take over the project without the original author.

In scope: voice and text conversations, appointment CRUD, patient registration, follow-up consultations, visit history, a REST API, and telephony through Asterisk.

Out of scope: payments and billing, prescriptions, a doctor-facing user interface, multiple clinics, and languages other than English.

Verification status (please read)

Automated checks (`test_logic.py`) cover the language-parsing code: intents, more than 30 spoken time formats, patient-ID extraction, names, yes/no and number choices. The database layer, conversation flows, speech engines and the telephony bridge were written and syntax-checked, but they were **not executed end to end** in the authoring environment, which had no MongoDB, microphone, internet or Asterisk. Treat the first deployment as a commissioning exercise and run the smoke test in section 17 before trusting it.

2. System at a glance

Item	Value
Agent persona	Asha, Sunrise Clinic (both configurable)
Interfaces	Phone (Asterisk), laptop microphone, keyboard, REST API
Speech to text	faster-whisper, model small.en by default
Text to speech	Piper, voice en_US-amy-medium (female)
Understanding	Keyword rules, regular expressions, dateparser, optional local LLM (Ollama, llama3.2:3b)
Database	MongoDB 7 through pymongo
Language and runtime	Python 3.10 or newer
Booking hours	8:00 AM to 6:00 PM in 30-minute slots (last slot starts 5:30 PM)
Closed days	Sundays by default
Advance booking	Up to 60 days
Time zone	Asia/Kolkata by default (set with the TIMEZONE variable)
Concurrency model	One thread and one Agent object per call
Seed data	5 doctors, 10 patients, 11 past visits, up to 5 sample appointments

3. Architecture

3.1 Layers

```
Caller (phone / laptop mic / keyboard)
        |
   IO adapter   (AudioSocketIO | MicIO | TextIO)
        |
   Speech layer (Whisper hears, Piper speaks)
        |
   Agent brain (agent.py)  <---->  Understanding (nlu.py)  --(fallback)-->
Local LLM
        |
   Booking rules (services.py)        REST API (api.py) uses the same rules
        |
   MongoDB (patients, doctors, appointments, visits)
```

3.2 Design principles

1. **One agent per call.** Each call gets its own Agent object holding its conversation state. Nothing is shared between calls except the database.
2. **Adapter pattern.** The Agent only ever calls `say(text)` and `listen()`. Three interchangeable adapters provide them, so the same brain works on a phone line, a microphone or a keyboard.
3. **Services are the single source of truth for rules.** The voice agent and the REST API both call `services.py`, so a rule is written once.
4. **The database has the final word on conflicts.** Unique indexes make double booking impossible even when two requests race.
5. **Rules first, LLM second.** Keyword and pattern rules handle almost everything quickly and predictably. A local LLM is consulted only when the rules cannot tell what the caller wants.
6. **Local and free.** After a one-time model download nothing needs the internet or a paid API. The one real cost is a public phone number, which is a rental from a telecom provider.

3.3 Life of a call

1. The call arrives. Asterisk streams the caller's audio to `telephony.py` (or the laptop mic or keyboard is used in test modes).

2. The Agent writes a sentence. Piper turns it into speech and the caller hears it.
3. The caller answers. The adapter records until the caller stops talking, and Whisper converts the audio to text.
4. `nlu.py` extracts the intent and details (day, time, doctor, patient ID).
5. The Agent decides what to ask or do next, calling `services.py` for rules and database access.
6. Steps 2 to 5 repeat until the caller says goodbye or stays silent.

Conversation state lives only in the running Agent object. After the call ends, the only thing that persists is what was saved in MongoDB.

3.4 How many agents, and is NLP used?

There is one Agent class, instantiated once per call. Whisper, Piper, the parser, the booking rules and the optional LLM are tools the agent uses, not separate agents. The system is a single-agent, goal-directed, tool-using design with a scripted dialogue state machine. It is not a multi-agent system and it is not a fully autonomous LLM agent. Every action on data goes through fixed functions, so the AI cannot invent or delete bookings.

Natural language processing is used in several forms: neural speech recognition (Whisper), neural speech synthesis (Piper), rule-based intent detection, entity extraction for dates, times, names and IDs, fuzzy string matching for doctor names, and an optional generative LLM fallback. No model is trained by this project; all models are pretrained.

4. Technology stack

Component	Choice	Why	Alternatives
Telephony	Asterisk with AudioSocket	Free PBX that streams call audio over TCP	FreeSWITCH, Pipecat
Speech to text	faster-whisper	Accurate, runs on CPU with int8	Vosk (lighter, less accurate)
Text to speech	Piper, voice Amy	Natural offline female voice	espeak-ng (robotic fallback built in), Coqui
Understanding	Regex, dateparser, difflib, optional Ollama	Fast, predictable, free	Tool-calling LLM

Component	Choice	Why	Alternatives
Database	MongoDB with pymongo	Flexible documents, unique partial indexes	PostgreSQL with exclusion constraints
API	FastAPI and uvicorn	Automatic interactive docs	Flask
Audio handling	sounddevice, soundfile, numpy	Record, play and resample audio	PyAudio
Time zones	zoneinfo and tzdata	Correct local time on every OS	pytz
Containers	Docker Compose (MongoDB only)	One command to start the database	Local MongoDB install

5. Repository layout

File	Responsibility
`config.py`	All settings: clinic name, hours, slot length, time zone, model paths, LLM settings, debug flag
`db.py`	MongoDB connection (timezone aware), index creation, and migration of old string-based records
`fmt.py`	Turns dates, times and lists into friendly spoken text
`services.py`	All business rules and all database reads and writes: patients, doctors, slots, appointments, conflicts, visits
`nlu.py`	Understanding: yes/no, intent, time and date parsing, patient ID, names, number choices, LLM fallback
`speech.py`	Whisper transcription, Piper synthesis, and the TextIO and MicIO adapters

File	Responsibility
`agent.py`	The conversation brain: greeting, identification, registration, and the book, check, reschedule, cancel, follow-up and history flows
`main.py`	Entry point for keyboard and microphone modes
`telephony.py`	TCP server for Asterisk AudioSocket and the AudioSocketIO adapter
`api.py`	FastAPI REST endpoints over the same services
`seed.py`	Creates doctors, 10 sample patients, sample visits and appointments
`test_logic.py`	Self-checks for the language parsing
`asterisk_extensions.conf`	Dialplan snippet that routes extension 100 to the agent
`docker-compose.yml`	Starts MongoDB 7 with a persistent volume
`requirements.txt`	Python dependencies
`models/`	Place the Piper voice files here

6. Data model

MongoDB database name: `clinic`. Four collections. Times are stored as real, timezone-aware datetimes, never as strings.

6.1 patients

Field	Type	Notes
patient_id	string	Six random digits plus the first letter of the first name and the first letter of the last name, for example 482913PL. Unique. The six-digit part is also kept unique so it can be used as a fallback lookup
first_name, last_name	string	Stored in title case

Field	Type	Notes
phone	string	Digits only; used for caller-ID matching on the last 10 digits
created_at	datetime	

6.2 doctors

Field	Type	Notes
doctor_id	string	D1 to D5 in the seed data. Unique
name	string	For example Dr. Meena Iyer
specialty	string	For example General Physician
keywords	list of strings	Words that route a caller to this doctor, such as fever, heart, skin, knee

6.3 appointments

Field	Type	Notes
appointment_id	string	Eight hex characters. Unique
patient_id, doctor_id, doctor_name	string	doctor_name is stored for fast reading
start_at, end_at	datetime	end_at is start_at plus the slot length
status	string	booked, cancelled or completed
type	string	regular or follow_up
reason	string or null	For follow-ups, the original complaint
follow_up_of	string or null	The visit_id this follow-up refers to
created_at, updated_at, cancelled_at	datetime	Audit timestamps

6.4 visits (medical history)

Field	Type	Notes
visit_id	string	Eight hex characters. Unique
patient_id, doctor_id, doctor_name	string	
visit_at	datetime	
complaint, diagnosis, notes	string	Free text recorded by the doctor
status	string	resolved, improving or ongoing
appointment_id	string or null	If set, that appointment is marked completed
created_at	datetime	

6.5 Indexes

Collection	Index	Purpose
patients	patient_id, unique	Fast and unique lookup
patients	phone	Caller-ID lookup
doctors	doctor_id, unique	
appointments	appointment_id, unique	
appointments	uniq_doctor_slot: (doctor_id, start_at), unique, only where status is booked	A doctor cannot have two active bookings at the same moment
appointments	uniq_patient_slot: (patient_id, start_at), unique, only where status is booked	A patient cannot be in two places at the same moment
visits	(patient_id, visit_at descending)	Fast latest-visit and history queries
visits	visit_id, unique	

The two slot indexes are partial: they apply only to documents whose status is booked. Cancelling an appointment changes its status, so its slot becomes free again while the record is kept.

6.6 Migration from version 1

Version 1 stored `date` and `time` as strings. On startup `init_indexes()` drops the old index named `doctor_id_1_date_1_time_1`, converts every old record to `start_at` and `end_at`, marks them regular, and removes the old fields. Running `python seed.py --reset` instead gives a clean start.

7. Business rules

Rule	Value	Where it is enforced
Booking hours	8 AM to 6 PM, 30-minute slots, last start 5:30 PM	`all_slots`, `valid_time` in services.py
Closed days	Sunday (CLOSED_WEEKDAYS)	`validate_date`
Past dates and times	Rejected; today's slots in the past are hidden	`validate_date`, `available_slots`
Advance limit	60 days	`validate_date`
No double booking of a doctor	One booked appointment per doctor per start time	`_check_slot`, then the uniq_doctor_slot index
No self-conflict for a patient	Overlap check against the patient's other booked appointments, with any doctor	`patient_conflict`, then the uniq_patient_slot index
Cancellation	Soft delete: status becomes cancelled, history is kept	`cancel`
Reschedule	Same checks as booking, excluding the appointment being moved; moving to the same time is refused	`reschedule`
Patient ID format	Six random digits, then first-name initial and last-name initial, uppercase	`create_patient`

Rule	Value	Where it is enforced
Follow-up	Linked to the latest visit and the same doctor by default; stored as type follow_up	`followup` in agent.py, `book` in services.py

8. Conversation design

8.1 Top-level flow

1. Greeting: Hello, and welcome to Sunrise Clinic. I'm Asha, your virtual assistant.
2. Identify the patient (section 8.2).
3. Menu loop: Asha asks how she can help. The caller can book, check, reschedule or cancel an appointment, book a follow-up consultation, or hear past visits.
4. The matching handler runs. Afterwards Asha asks whether there is anything else. A no, or a phrase such as goodbye or that's all, ends the call politely.

8.2 Identification

1. If caller-ID is available and matches a registered phone number, Asha asks whether she is speaking with that person.
2. Otherwise she asks whether the caller already has a patient ID. If yes, the caller says six digits followed by two letters, with up to three attempts.
3. If the exact ID is not found, the system falls back to the six digits. If exactly one patient matches, Asha confirms the name before continuing. This tolerates a misheard letter.
4. If the ID cannot be found, Asha offers registration.

8.3 Registration

Asha asks for the first name, the last name and a ten-digit mobile number (or offers to use the caller-ID number), then confirms the details. After saving, she reads the new patient ID aloud character by character, repeats it once, and asks the caller to note it down.

8.4 Booking (create)

1. Choose a doctor by name, specialty or health concern (section 9.6).
2. Ask for a day and time. A caller may give both in one sentence.

3. Validate the date. Compute free slots as the slots free for the doctor and also free for the patient.
4. If no time was given, read the free slots. If the requested time is taken, outside hours, or conflicts with the patient's own appointment, explain why and offer the three nearest free times.
5. Read the details back and ask for a yes. On yes, save through `services.book`.

8.5 Check (read)

Asha lists the caller's upcoming booked appointments in time order, numbered, with doctor, day and time. Follow-ups are labelled.

8.6 Reschedule (update)

The caller picks an appointment (automatic if there is only one), chooses a new day and time through the same slot logic with the appointment itself excluded, confirms, and `services.reschedule` applies the change.

8.7 Cancel (delete)

The caller picks an appointment, confirms, and `services.cancel` marks it cancelled.

8.8 Follow-up consultation

Asha finds the caller's latest visit and reads it back: date, doctor, complaint and whether the problem was resolved, improving or still ongoing. She offers a follow-up with the same doctor, or another doctor if the caller prefers. The booking is saved as type follow_up with the original complaint as the reason and a link to the original visit. If there is no history, she offers a regular appointment instead.

8.9 Past visits

Asha reads up to the last three visits with date, doctor, complaint, diagnosis (if recorded) and status.

8.10 Retry and error policy

Situation	Behaviour
Silence or nothing transcribed	Asks again, up to three times, then ends the call politely
Unclear yes or no	Asks the caller to say yes or no, up to three times, then treats it as no
Unrecognised request	Repeats the list of things she can do
Unrecognised doctor	Asks again, up to three times, then gives up on that request
Date or slot problems	Explains the reason and asks for another day or time, up to five rounds
Caller hangs up	The connection error is caught and the call ends quietly

8.11 Spoken output conventions

Times are spoken as 8:00 AM or 5:30 PM. Dates are spoken as Friday, 16 October. Patient IDs are spoken one character at a time, separated by commas, so the voice does not run them together.

9. Language understanding (nlu.py)

Understanding is deliberately rule-based first. Rules are fast, free, predictable and easy to debug. The local LLM is a safety net, not the engine.

9.1 Yes and no

`yes_no(text)` looks at the words spoken. Positive words (yes, yeah, sure, correct, okay and similar) without any negative word give yes. Negative words (no, nope, not, wrong and similar) without any positive word give no. Anything else is undecided, and the Agent asks again.

9.2 Intent detection

`intent(text)` checks the rules in a fixed order and returns the first match. Order matters: a sentence such as I want to cancel my appointment contains both cancel and appointment, and the more specific intent must win.

Priority	Intent	Triggered by words like
1	exit	bye, goodbye, hang up, that's all, nothing else, I'm done
2	delete	cancel, delete, remove, drop
3	update	reschedule, change, move, update, modify, shift, postpone, edit
4	followup	follow-up, recheck, review, not getting better
5	history	history, past visits or records, previous visits or records, medical records
6	availability	available, free slot, open slot, slots
7	read	check, view, list, existing, my appointments, status, upcoming
8	create	book, schedule, make, new, create, arrange, need, want, see a doctor, appointment

If nothing matches, `_llm_json` asks the local Ollama model to classify the sentence. The call uses JSON output mode, temperature 0 and a 20-second timeout. Any failure (Ollama not installed, not running, slow or returning bad JSON) quietly returns unknown, and Asha asks the caller to choose from her list. The allowed intents are create, read, update, delete, availability, followup, history and exit. The availability intent is handled by the booking flow, which reads out free slots first.

9.3 Time parsing: from spoken words to a typed value

This was the most error-prone part of the system, so it is a pipeline:

1. Whisper produces text such as 8.30 AM tomorrow.
2. `parse_time` notes any period-of-day word (morning, afternoon, evening).

3. It tries these patterns in order and uses the first valid match: half or quarter past or to an hour; hour and minute with a colon or dot (8:30, 8.30); three or four digits with am or pm (830am); hour, space, minute and am or pm (8 30 am); a word hour and word minute (eight thirty); hour with am or pm (8am, 8 a.m.); hour with o'clock; hour followed by in the morning, afternoon or evening; noon or midday; and finally a bare at 3.
4. Number words are converted (eight, thirty, forty five, oh five). Invalid values such as hour 30 are rejected and the next pattern is tried.
5. The meridiem is decided: an explicit am or pm always wins. Otherwise a morning cue keeps the hour, an afternoon or evening cue adds 12 to hours below 12, and with no cue the hours 1 to 7 are taken as PM (clinic hours), 8 to 11 stay AM, and 12 is noon.
6. The result is always a zero-padded 24-hour string such as 08:00 or 17:30.
7. The time phrase is removed from the text, and what is left goes to the date parser. This stops a time such as 8.30 being mistaken for a date.

Spoken	Result
8am, 8 a.m., 8 AM	08:00
8:30AM, 8.30 AM, 830 am, 8 30 am	08:30
eight thirty, half past eight	08:30
quarter to nine	08:45
eight oh five am	08:05
ten in the morning	10:00
two thirty in the afternoon	14:30
at 3, 3 o'clock	15:00
noon	12:00

Because bookings are in 30-minute slots, a time such as 8:15 is valid as a time but is not a slot. Asha explains the hours and offers the nearest free slots.

9.4 Date parsing and the type conversion

`parse_date` uses dateparser's `search_dates` in English, preferring future dates, with today taken in the clinic's time zone. It understands tomorrow, Monday, next Friday and October twentieth. The date and the time string are then combined in one place,

`services.make_start(date, hhmm)`, which builds a timezone-aware datetime. That datetime is what MongoDB stores as `start_at`. Strings are never stored.

With DEBUG on (the default), every utterance prints a line such as `[heard] '8 am tomorrow' -> date=2026-10-19 time=08:00`. This is the first thing to check when a caller reports a misunderstanding.

9.5 Patient ID, names and numbers

- **Patient ID:** `extract_patient_id` collects the first six digits (digits or number words such as four, eight, oh), then up to two letters. It understands letter homophones such as kay for K and are for R, and ignores filler words. It returns the digits and letters separately so the lookup can fall back to the digits alone.
- **Phone numbers:** `digits_only` keeps digits and converts number words.
- **Names:** `extract_name` drops fillers such as my, name, is, this and mister, and returns the first remaining word with a capital letter.
- **Choices:** `parse_choice` reads the first, second, third, a digit, or last from a spoken list.

9.6 Doctor matching

The code is `match_doctor(text)` in services.py. Each doctor document carries a `keywords` list, seeded from the DOCTORS list in seed.py. It works in two passes:

1. **Keyword pass.** If any of a doctor's keywords appears as a whole word in what the caller said, that doctor is returned. A caller who says skin rash is routed to the dermatologist.
2. **Fuzzy pass.** If there is no keyword hit, each spoken word of three or more letters is compared with the tokens of each doctor's name and specialty using `difflib.get_close_matches` with a cutoff of 0.8. This tolerates transcription slips such as Menna for Meena.

Because keywords live in the database, adding a doctor or a new symptom keyword needs no code change.

10. Speech layer (speech.py)

10.1 Speech to text

`load_models()` loads faster-whisper lazily on first use, on CPU with int8 quantisation. `transcribe(audio)` receives 16 kHz mono audio, uses English, beam size 1 and the built-in voice-activity filter. A lock serialises transcription so that concurrent calls do not corrupt each other. The model name comes from WHISPER_MODEL (small.en by default; base.en is faster on weak machines; medium or distil-large-v3 is more accurate). The first run downloads the model, roughly 500 MB for small.en.

10.2 Text to speech

`synth(text)` returns audio samples and a sample rate. It runs Piper as a subprocess (`python -m piper -m <model> -f <wav>`), sending the text on standard input, reads the resulting WAV and deletes it. If the Piper model file is missing or Piper fails, it falls back to espeak-ng with a female voice variant, which sounds robotic. If neither works it raises a clear error. Each sentence starts a new Piper process, which adds roughly a second of latency. Keeping Piper loaded is a listed improvement.

10.3 The three adapters

Adapter	Where	say(text)	listen()
TextIO	speech.py	Prints the sentence	Reads a typed line with input()
MicIO	speech.py	Prints, synthesises with Piper, plays through the speakers	Records from the microphone at 16 kHz in 100 ms blocks, starts when volume (RMS) passes 0.012, stops after 1.2 seconds of silence or 12 seconds in total, then transcribes

Adapter	Where	say(text)	listen()
AudioSocketIO	telephony.py	Synthesises, resamples to 8 kHz, sends 20 ms frames of 320 bytes to Asterisk, then discards audio that arrived while speaking	Reads frames from Asterisk, uses the same volume and silence logic (1.0 second silence, 12 seconds maximum), resamples 8 kHz to 16 kHz and transcribes

10.4 Echo and turn-taking

MicIO is half-duplex: it does not record while it is speaking. AudioSocketIO keeps the caller's audio from piling up by draining and discarding incoming frames after each spoken sentence, parsing frames properly so the stream never falls out of alignment. Callers cannot interrupt Asha mid-sentence (no barge-in).

10.5 Tuning

If a noisy phone line is cut off too early or never detected, adjust the volume threshold (`threshold` or `thr`, default 0.012) and the silence length (`silence_seconds` or `silence_s`). Lower thresholds pick up quieter speech but also more noise.

11. Telephony integration (telephony.py and Asterisk)

11.1 Call path

Caller, then the telephone network or a SIP softphone, then Asterisk, which runs the AudioSocket application and connects over TCP to `telephony.py` on port 9092. Audio flows both ways over that one connection.

11.2 AudioSocket framing

Every message is one byte of type, two bytes of payload length (big-endian), then the payload. The types used are 0x01 (call UUID, sent first), 0x10 (audio: signed 16-bit, 8 kHz, mono) and 0x00 (hang-up). `telephony.py` accepts connections, starts one daemon thread per call, reads the UUID frame, builds an `AudioSocketIO` and runs a fresh `Agent`. A hang-up frame or a closed socket raises a connection error that ends the call cleanly.

11.3 Dialplan

Add this to extensions.conf (the repository file is asterisk_extensions.conf):

```
[from-internal]
exten => 100,1,NoOp(Clinic voice agent)
 same => n,Answer()
 same => n,Set(UUID=${SHELL(uuidgen | tr -d '\n')})
 same => n,AudioSocket(${UUID},127.0.0.1:9092)
 same => n,Hangup()
```

11.4 Setup steps

1. Install Asterisk 18 or newer and confirm the AudioSocket modules are loaded.
2. Add the dialplan above and create a PJSIP endpoint for a test extension.
3. Reload with `asterisk -rx "core reload"`.
4. Start the bridge with `python telephony.py`.
5. For a free demo, register the Linphone app to Asterisk on the same network and dial 100.

11.5 Real phone numbers and caller-ID

The software is free, but a public phone number is a rental from a telecom provider (a SIP trunk or DID). Route its inbound calls to extension 100. The Agent already supports caller-ID recognition through its `caller_phone` argument (use `--caller` in test modes), but the telephony bridge does not yet pass the caller's number from Asterisk. Wiring that through (for example with a channel variable or ARI) is a recommended improvement.

12. Where each operation lives

Operation	Conversation (agent.py)	Rules and database (services.py)	MongoDB action	REST endpoint
Create appointment	book, _book_with	book	insert_one	POST /appointments
Read appointments	read, _pick	upcoming	find, sorted by start_at	GET /patients/{pid}/appointments
Update (reschedule)	update	reschedule	update_one	PUT /appointments/{aid}

Operation	Conversation (agent.py)	Rules and database (services.py)	MongoDB action	REST endpoint
Delete (cancel)	delete	cancel	update_one, status set to cancelled	DELETE /appointments/{aid}
Register patient	register	create_patient	insert_one	POST /patients
Find patient	identify	find_patient, find_patient_by_phone, get_patient	find_one, find	GET /patients/{pid}
Free slots	choose_slot	available_slots, patient_busy_times	find	GET /doctors/{id}/slots
Record a visit	not by voice	add_visit	insert_one, and marks the appointment completed	POST /visits
Read history	followup, history	latest_visit, history	find, newest first	GET /patients/{pid}/history
Doctor's day list	not by voice	schedule	find, with the latest visit per patient	GET /doctors/{id}/schedule

The direction of calls is always agent.py to services.py to MongoDB. The agent never talks to the database directly.

13. Conflict handling and concurrency

13.1 Three layers of protection

Layer	File and functions	Job
Conversation	agent.py, `choose_slot`	Offer only times free for both doctor and patient, and explain a clash in speech

Layer	File and functions	Job
Business rules	services.py, `_check_slot`, `patient_conflict`, `patient_busy_times`, `conflict_message`, used by `book` and `reschedule`	Enforce the rules for every caller, including the REST API
Database	db.py, `init_indexes`: indexes uniq_doctor_slot and uniq_patient_slot	Make a clash impossible even when requests race

Each layer covers the weakness of the others. The agent alone would be bypassed by the API. The services check alone can be beaten by two simultaneous requests that both see a free slot. The database alone cannot explain anything to a caller. In short: the agent makes it friendly, the services make it correct, and the database makes it unbreakable.

13.2 The same patient, two doctors, same time

When choosing a slot, the patient's own booked times are removed from the offered list. If the caller asks for such a time, Asha says: You already have an appointment with the named doctor at that time on that day, so I can't book two appointments at the same time, and offers the nearest free times. `book` and `reschedule` re-check with fresh data, and the uniq_patient_slot index is the final guard. When rescheduling, the appointment being moved is excluded from its own check.

13.3 Two callers booking the same slot at the same moment

1. Two calls run in two threads with two Agent objects.
2. Both see the slot as free, because neither has saved yet.
3. Both callers confirm, and both reach `insert_one`.
4. MongoDB processes the writes one after the other, and uniq_doctor_slot accepts only one.
5. The loser receives a duplicate-key error. `_dup_message` inspects which index was violated and returns either Sorry, someone just took that slot, or the patient-clash message.
6. The caller is asked whether there is anything else and can book again.

The key point is that the database is the single source of truth. The Python check is a courtesy that can be stale; the index cannot be beaten by a race.

13.4 Known gaps

- After losing a race the agent does not automatically re-offer slots; the caller has to ask to book again.
- The unique indexes work on exact start times. If SLOT_MINUTES is changed while old appointments exist, doctor overlaps between different lengths are not caught by the database. Patient overlaps are still checked by start and end time in `patient_conflict`.

14. REST API reference (api.py)

Start with `uvicorn api:app --reload`. Interactive documentation is at http://localhost:8000/docs. Dates are YYYY-MM-DD and times are HH:MM in 24-hour form.

Method and path	Purpose	Body or query
GET /doctors	List doctors	none
GET /doctors/{doctor_id}/slots	Free slots for a day	query: day
GET /doctors/{doctor_id}/schedule	Booked appointments for a day with each patient's latest visit	query: day
POST /patients	Register a patient and get a patient ID	first_name, last_name, phone (optional)
GET /patients/{pid}	Fetch a patient	none
GET /patients/{pid}/appointments	Upcoming appointments	none
GET /patients/{pid}/history	Visit history, newest first	query: limit (default 10)
POST /appointments	Book	patient_id, doctor_id, day, time
PUT /appointments/{aid}	Reschedule	patient_id, day, time
DELETE /appointments/{aid}	Cancel	query: patient_id

Method and path	Purpose	Body or query
POST /visits	Record a visit, optionally completing an appointment	patient_id, doctor_id, complaint, diagnosis, notes, status, appointment_id

Status codes: 409 when a booking rule is broken (slot taken, outside hours, self-conflict), 404 when a patient or appointment is not found, 400 for an invalid visit status. The API has **no authentication**; it must not be exposed publicly as it is (see section 20).

15. Configuration reference (config.py)

Setting	Default	Meaning	Environment variable
CLINIC_NAME	Sunrise Clinic	Spoken in the greeting	none
AGENT_NAME	Asha	The assistant's name	none
MONGO_URI	mongodb://localhost:27017	Database connection	MONGO_URI
DB_NAME	clinic	Database name	none
OPEN_HOUR, CLOSE_HOUR	8, 18	Booking window	none
SLOT_MINUTES	30	Slot length	none
CLOSED_WEEKDAYS	Sunday (6)	Closed days; use an empty set to open every day	none
MAX_ADVANCE_DAYS	60	How far ahead callers can book	none
WHISPER_MODEL	small.en	Speech recognition model	WHISPER_MODEL
PIPER_MODEL	models/en_US-amy-medium.onnx	Voice file	PIPER_MODEL
OLLAMA_URL	http://localhost:11434/api/generate	Local LLM endpoint	OLLAMA_URL
OLLAMA_MODEL	llama3.2:3b	Local LLM model	OLLAMA_MODEL

Setting	Default	Meaning	Environment variable
TIMEZONE	Asia/Kolkata	Clinic local time	TIMEZONE
DEBUG	on	Print what was heard and how it was parsed	DEBUG (set to 0 to turn off)

16. Setup and runbook

16.1 Prerequisites

Python 3.10 or newer, Docker, and internet access once to download models. On Linux install the audio and speech helpers first with `sudo apt install libportaudio2 espeak-ng`. On Windows the `tzdata` package (already in requirements.txt) is needed for time zones.

16.2 First-time setup

```
docker compose up -d
python -m venv venv
source venv/bin/activate          (Windows: venv\Scripts\activate)
pip install -r requirements.txt
```

Download the two Piper voice files into the `models/` folder:

- https://huggingface.co/rhasspy/piper-voices/resolve/main/en/en_US/amy/medium/en_US-amy-medium.onnx
- https://huggingface.co/rhasspy/piper-voices/resolve/main/en/en_US/amy/medium/en_US-amy-medium.onnx.json

Optional, for smarter intent fallback: install Ollama from ollama.com and run `ollama pull llama3.2:3b`.

Then load the sample data and run the self-checks:

```
python seed.py --reset
python test_logic.py
```

`seed.py --reset` empties the four collections, creates the five doctors, ten patients (printing their IDs, which you need for testing), eleven past visits and a few appointments on upcoming working days. Run it without `--reset` to add data only if the collections are empty.

16.3 Ways to run

Mode	Command	Use
Keyboard	`python main.py --mode text`	Fastest way to test the logic with no audio
Keyboard with caller-ID	`python main.py --mode text --caller 9840011101`	Test the recognised-caller path
Microphone and speakers	`python main.py --mode voice`	Laptop acts as the phone; downloads the Whisper model on first run
REST API	`uvicorn api:app --reload`	Staff tools and integrations; docs at /docs
Phone calls	`python telephony.py`	Needs Asterisk (section 11)

16.4 Inspecting data

Use `mongosh` against `mongodb://localhost:27017`, then `use clinic` and queries such as `db.appointments.find().pretty()`, `db.visits.find()` and `db.appointments.getIndexes()`.

17. Testing

17.1 What exists

`test_logic.py` asserts the behaviour of the language code: eight intents, more than 30 spoken time formats, patient-ID extraction from digits and words, name extraction, yes/no and numbered choices, plus a combined date and time check. It needs the dateparser package for the date check.

17.2 What does not exist yet

There are no automated tests for services.py, the conversation flows, the speech engines or the telephony bridge. This is the biggest quality gap in the project (see section 19).

17.3 Commissioning smoke test

Run these by hand in text mode, then repeat a few in voice mode, before relying on the system:

1. `seed.py --reset` prints ten patient IDs and the doctor and appointment counts.
2. Book an appointment with a seeded ID, then confirm it in mongosh with start_at and end_at as real dates.
3. As a second patient, try to book the same doctor and time. It must be refused.
4. As the same patient, try to book a different doctor at the same time. It must be refused with the explanation that two appointments cannot overlap.
5. Reschedule one appointment onto the time of another of the same patient. It must be refused.
6. Cancel an appointment, then book the freed slot. It must succeed and the cancelled record must remain with status cancelled.
7. Try a Sunday, 7:00 AM, 6:00 PM and a past date. Each must be refused with a clear reason.
8. Use the first seeded patient for a follow-up consultation and for past visits.
9. In voice mode say 8am, 8.30 AM and half past eight, and check the `[heard]` lines.
10. Race test: in two terminals prepare the same slot for two patients, confirm at the same moment, and check that exactly one booking exists.

17.4 Recommended automated tests

Use pytest. Test services.py against a real MongoDB in a test container, because the partial unique indexes are central to correctness and a mock may not reproduce them faithfully. Add tests for the conversation flows by feeding scripted answers through a fake IO adapter, and for the REST API with FastAPI's test client.

18. Troubleshooting

Symptom	Likely cause	Fix
ServerSelectionTimeoutError	MongoDB is not running or MONGO_URI is wrong	Run `docker compose up -d`; check the URI
ZoneInfoNotFoundError	tzdata missing (common on Windows)	`pip install tzdata`
No TTS found error or robotic voice	Piper model files missing or PIPER_MODEL path wrong	Put both files in models/, or set PIPER_MODEL; espeak-ng is only a fallback
PortAudio or sounddevice error	Audio library or microphone permission missing	Install libportaudio2; allow microphone access
First voice start is very slow	Whisper model downloading	Wait once, or use WHISPER_MODEL=base.en
Asha cuts the caller off or never hears them	Volume threshold or silence length unsuitable	Tune `threshold` and `silence_seconds` (section 10.5)
A time or date is misunderstood	Transcription or parsing issue	Read the `[heard]` debug line; send it with the bug report
Names are spelled wrongly	Speech recognition limit, worse on phone audio	See the improvement plan in section 21.3
Index creation fails at startup	Existing data violates a unique index, for example duplicate bookings from old data	Inspect the duplicates, or run `seed.py --reset` on a test system
Intent fallback never works	Ollama not installed or not running	Harmless; install and start Ollama if you want the fallback

Symptom	Likely cause	Fix
Phone call connects but is silent	AudioSocket module not loaded, bridge not running, or port 9092 blocked	Check Asterisk modules, run `python telephony.py`, check the firewall
Choppy phone audio	CPU overloaded by speech models	Smaller models, more CPU, or a GPU

19. Known limitations and risks

1. **Speech accuracy.** Names are misheard roughly half the time in informal testing reports, and phone-quality audio makes it worse. Mitigations are listed in section 21.3.
2. **Not fully tested.** Only the language parser has automated checks, and the full stack has not been run end to end by the author (section 17).
3. **Weak identity check.** A patient ID alone unlocks a patient's appointments and medical history.
4. **No API authentication.** Anyone who can reach the API can read and change data.
5. **Caller-ID is not passed from Asterisk** to the Agent yet.
6. **Latency.** Piper starts a new process per sentence, and Whisper transcriptions are serialised by a lock, so many simultaneous calls will queue.
7. **No barge-in.** Callers cannot interrupt Asha while she speaks.
8. **English only,** and the optional LLM is small and may misjudge unusual phrasing.
9. **Fixed slots.** All appointments are 30 minutes; there are no doctor-specific schedules, breaks, leave or shift patterns.
10. **No lost-race recovery.** After losing a race for a slot the agent does not re-offer alternatives automatically.
11. **Visits are recorded only through the API.** There is no doctor-facing screen.
12. **Changing slot length** after bookings exist needs care (section 13.4).
13. **Timezone assumption.** One clinic time zone is used for everything.

20. Security and privacy

The system handles personal and health-related information, so treat it accordingly. Check the privacy and health-data laws that apply where you operate; this document is not legal advice.

Current state: the REST API has no authentication; the docker-compose MongoDB has no username or password; records are not encrypted at rest; DEBUG mode prints transcripts to the console; calls are not recorded; callers are not told that an AI is answering.

Recommended before real patients use it:

- Add a second factor before sharing health details: an SMS one-time code to the registered number, or a date-of-birth check.
- Add authentication and TLS to the API, and keep it off the public internet.
- Enable MongoDB authentication, restrict its network access and encrypt storage and backups.
- Turn DEBUG off in production, or redact transcripts in logs, and define a retention period.
- Announce at the start of each call that an automated assistant is answering, and obtain consent if calls or transcripts are stored.
- Limit what is read aloud: confirm identity first, and offer to send sensitive details by a secure channel instead.

21. Scaling and roadmap

21.1 Scaling the architecture

Area	Today	Direction
Speech models	Whisper and Piper on one machine, Piper started per sentence	GPU or smaller models, keep Piper loaded, streaming recognition, separate speech worker services shared by many calls
Call handling	One bridge process, one thread per call	Several bridge servers behind a load balancer or Asterisk or FreeSWITCH cluster

Area	Today	Direction
Call state	Inside each Agent object in memory	Store in Redis so any worker can continue a call
Database	Single MongoDB container	Replica set for failover, sharding only at very large size; the unique indexes keep working
Deployment	Manual	Docker images per service, Kubernetes with autoscaling
Visibility	Console prints	Metrics and dashboards (Prometheus, Grafana), structured logs, alerts on failed calls and slow responses

21.2 Feature roadmap

- Doctor-specific working hours, breaks and leave; variable appointment lengths.
- SMS or WhatsApp confirmations and reminders; outbound reminder calls using the same agent.
- Waiting list that fills cancelled slots automatically.
- Doctor dashboard (for example React over the existing API) to record visits and see the day's list with patient history.
- Calendar sync (Google Calendar or similar).
- Multilingual support: multilingual Whisper, matching Piper voices, translated prompts and dateparser languages.
- Barge-in so callers can interrupt, and a hand-off to a human receptionist.
- Analytics on no-shows, busy hours and call outcomes.

21.3 Improving name and speech accuracy

The misspelled names come from the listening side (Whisper), not from Piper. An LLM cannot recover spelling that the audio never made clear, so it helps most when it has something to check against. In order of value for effort:

1. Pass hint words (doctor names, common patient names) to Whisper through its prompt or hotword options.
2. Use a larger model such as medium or distil-large-v3 if the hardware allows.

3. For recognised patients, compare what was heard with the name on file using fuzzy and phonetic matching.
4. For new patients, ask them to spell their name letter by letter and read it back.
5. Use Whisper's confidence score to decide when to ask again instead of guessing.
6. Pass the caller-ID from Asterisk so most returning patients need no name at all.
7. As a last stage, let a local LLM correct low-confidence transcripts, given the question that was asked and the candidate names from the database. The LLM only suggests; code validates against the database and the caller confirms. Expect roughly one to three seconds of extra delay on CPU, so use it only when confidence is low.

Measure each change by recording a fixed set of test calls and comparing name accuracy before and after.

22. Operations

Topic	Guidance
Backups	Schedule `mongodump` (or volume snapshots) of the clinic database and test restores
Monitoring	Watch process health for `telephony.py`, MongoDB availability, speech latency and the number of failed or abandoned calls
Logs	Keep structured logs without full transcripts; DEBUG output is for development
Upgrades	Starting the app runs `init_indexes()`, which creates indexes and migrates old records, so back up before upgrading
Model changes	Change WHISPER_MODEL, PIPER_MODEL or OLLAMA_MODEL through environment variables and restart
Resetting a test system	`python seed.py --reset` (never on production data)

Topic	Guidance
Deployment checklist	Authentication on, DEBUG off, MongoDB secured, backups scheduled, time zone set, models downloaded, smoke test passed, AI-answering announcement in place

23. Knowledge-transfer kit

23.1 Suggested onboarding path

1. Read sections 3, 5 and 12 of this document.
2. Start MongoDB, seed the data and run `python main.py --mode text` as a patient. Try booking, rescheduling, cancelling, follow-up and history.
3. Read services.py first (rules and data), then agent.py (conversation), then nlu.py (understanding).
4. Read speech.py and telephony.py to see how audio is turned into text and back.
5. Run the smoke test in section 17.3 and write your first automated test for `book`.

23.2 Extension recipes

Task	What to change
Add a doctor or a symptom keyword	Insert or update a document in the `doctors` collection (or the DOCTORS list in seed.py). No code change
Change opening hours	OPEN_HOUR and CLOSE_HOUR in config.py. Existing appointments are not moved
Open on Sundays	Set CLOSED_WEEKDAYS to an empty set
Change the clinic or assistant name	CLINIC_NAME and AGENT_NAME in config.py
Change the voice	Download another Piper voice and set PIPER_MODEL
Change the LLM	Set OLLAMA_MODEL

Task	What to change
Add a new intent	Add a rule to `RULES` and the name to `INTENTS` in nlu.py, write a handler method in agent.py, and add it to the handlers dictionary and the menu prompt
Add a language	Use a multilingual Whisper model, add a Piper voice for the language, translate the prompts, and set dateparser languages
Change slot length	SLOT_MINUTES, but read section 13.4 first

23.3 Design decisions and why

- **Why rules before an LLM?** Small local models are unreliable at precise details such as dates and IDs. Rules are fast, free and testable. The LLM covers the gaps.
- **Why does the database enforce conflicts too?** Python checks can be stale when calls overlap. Only a database constraint is race-proof.
- **Why partial unique indexes?** They apply only to booked appointments, so cancelled records stay for history while their slots become free.
- **Why soft delete?** It keeps an audit trail and supports reporting.
- **Why store real datetimes, not strings?** One conversion point, correct sorting and range queries, and no timezone ambiguity.
- **Why the adapter pattern?** The conversation logic is written once and works over keyboard, microphone and phone.
- **Why threads, one per call?** Simple and adequate at small scale; the scaling plan moves state out of memory.
- **Why Whisper and Piper?** Both are open source, run offline and give good quality without per-call fees.

23.4 Common review questions

- **Which line prevents double booking?** The `uniq_doctor_slot` index in `init_indexes` (db.py), backed by `_check_slot` in `book` (services.py).
- **How does a time like 8am become a stored value?** nlu.py produces 08:00, `make_start` combines it with the date into a timezone-aware datetime, and MongoDB stores it as `start_at`.

- **How does doctor matching work?** Whole-word keyword match first, then fuzzy name matching (section 9.6).
- **How many agents?** One Agent per call; the other parts are tools.
- **Is NLP used?** Yes: neural speech recognition and synthesis, rule-based intent and entity extraction, fuzzy matching and an optional LLM.
- **What happens if two calls take the same slot?** One succeeds, the database rejects the other, and that caller hears that the slot was just taken.

23.5 Glossary

Term	Meaning
STT, ASR	Speech to text; here faster-whisper
TTS	Text to speech; here Piper
NLU	Natural language understanding: finding intent and details in text
Intent	What the caller wants to do, such as book or cancel
Slot	Either a bookable time period, or a detail to extract such as a date; context makes clear which
RMS	A measure of audio volume used to detect speech and silence
VAD	Voice activity detection: deciding when someone is speaking
PBX	A private telephone exchange; Asterisk is one
SIP trunk, DID	A rented connection and phone number from a telecom provider
AudioSocket	Asterisk’s way of streaming call audio over TCP
Unique partial index	A database rule that forbids duplicates only among documents matching a condition
Soft delete	Marking a record as cancelled instead of erasing it
Race condition	A bug where two simultaneous actions interfere because each checked before the other finished

Term	Meaning
Adapter pattern	Hiding different devices behind one common interface

Appendix A. Sample call

Asha: Hello, and welcome to Sunrise Clinic. I'm Asha, your virtual assistant.

Asha: Do you already have a patient ID with us?

Caller: Yes.

Asha: Please say your patient ID: six digits followed by two letters.

Caller: Four eight two nine one three, P L.

Asha: Thank you, Priya. How can I help you today? You can book, check, reschedule or cancel an appointment, book a follow-up consultation, or hear your past visits.

Caller: I want to book an appointment.

Asha: Which doctor would you like to see? We have Dr. Meena Iyer, General Physician; Dr. Arjun Rao, Cardiologist; and so on. You can also tell me a specialty or a health concern.

Caller: I have a skin rash.

Asha: Which day would you like, and at what time?

Caller: Monday at eight thirty in the morning.

Asha: Shall I book Dr. Fatima Khan on Monday, 19 October at 8:30 AM?

Caller: Yes.

Asha: Done! Your appointment with Dr. Fatima Khan on Monday, 19 October at 8:30 AM is confirmed. Is there anything else I can help you with?

Caller: No, thank you.

Asha: Thank you for calling Sunrise Clinic. Take care. Goodbye!

A conflict example

Caller: Book the cardiologist on Monday at 8:30 AM.

Asha: You already have an appointment with Dr. Fatima Khan at 8:30 AM on Monday, 19 October, so I can't book two appointments at the same time. The closest free times are 8:00 AM, 9:00 AM and 9:30 AM. Which time would you like?

Appendix B. Function index

Module	Key functions and classes
services.py	all_slots, valid_time, validate_date, available_slots, nearest, make_start, list_doctors, get_doctor, match_doctor, create_patient, get_patient, find_patient, find_patient_by_phone, patient_conflict, patient_busy_times, conflict_message, book, upcoming, reschedule, cancel, add_visit, history, latest_visit, schedule
nlu.py	yes_no, intent, parse_time, parse_date, parse_datetime, extract_patient_id, digits_only, extract_name, parse_choice
agent.py	Agent (run, identify, register, choose_doctor, choose_slot, book, read, update, delete, followup, history), Hangup
speech.py	load_models, transcribe, synth, TextIO, MicIO
telephony.py	AudioSocketIO (frame, say, listen), handle, main loop
db.py	get_db, init_indexes, migration of legacy records
fmt.py	fmt_date, fmt_time, fmt_dt, join_list
api.py	FastAPI app and the endpoints in section 14